

Software Testing and Quality Assurance

Theory and Practice

Chapter 7

System Integration Testing

The Concept of Integration Testing

- A software module is a self-contained element of a system
- Modules are individually tested commonly known as *unit testing*
- Next major task is to put the modules, i.e., pieces together to construct the complete system
- Construction of a working system from the pieces is not a straightforward task because of numerous *interface errors*
- The objective of *system integration testing* (SIT) is to build a “working” version of the system
 - Putting modules together in an incremental manner
 - Ensuring that the additional modules work as expected without disturbing the functionalities of the modules already put together
- *Integration testing* is said to be complete when
 - The system is fully integrated together
 - All the test cases have been executed
 - All the severe and moderated defects found have been fixed

The Concept of Integration Testing

The major advantages of conducting SIT are as follows:

- Defects are detected early
- It is easier to fix defects detected earlier
- We get earlier feedback on the health and acceptability of the individual modules and on the overall system
- Scheduling of defect fixes is flexible, and it can overlap with development.

System integration testing is a systematic technique for assembling a software system while conducting tests to uncover errors associated with interfacing.

Large, complex software products can go through several iterations of build-and-test cycles before they are fully integrated.

DIFFERENT TYPES OF INTERFACES AND INTERFACE ERRORS

An interface between two modules allows one module to access the service provided by the other.

Three common paradigms for interfacing modules are as follows:

Procedure Call Interface: A procedure in one module calls a procedure in another module. The caller passes on control to the called module. The caller can pass data to the called procedure, and the called procedure can pass data to the caller while returning control back to the caller.

Shared Memory Interface: A block of memory is shared between two modules. The memory block may be allocated by one of the two modules or a third module. Data are written into the memory block by one module and are read from the block by the other.

Message Passing Interface: One module prepares a message by initializing the fields of a data structure and sending the message to another module. This form of module interaction is common in client–server-based systems and web-based systems.

DIFFERENT TYPES OF INTERFACES AND INTERFACE ERRORS

Element	Description
Module 1 (Caller)	CheckoutController (Handles user interaction)
Module 2 (Called)	InventoryService (Manages stock levels)
Interface Channel	Direct function/method invocation (e.g., in Python, Java, or C++)
Interaction	The CheckoutController calls a method in the InventoryService module. 1. Caller passes control: CheckoutController calls InventoryService.reserve_stock(sku, quantity). 2. Called performs service: InventoryService checks and updates the database. 3. Called returns data & control: InventoryService returns True (success) or False (out of stock) and gives control back to the CheckoutController.
Control/Data Flow	
Testing Focus	Testing must ensure the correct data types are passed and that the return value is accurately interpreted (e.g., if the service returns False, the controller displays the correct "Out of Stock" message).

DIFFERENT TYPES OF INTERFACES AND INTERFACE ERRORS

Element	Description
Module 1 (Writer)	MarketDataFeedProcessor (Pulls real-time prices from an exchange)
Module 2 (Reader)	TradingStrategyEngine (Calculates buy/sell signals)
Interface Channel	A dedicated, mutually accessible Shared Memory Buffer (a specific address in RAM).
Interaction	The two modules do not call each other; they access the shared memory concurrently.
Control/Data Flow	1. Writer: The Processor writes the latest stock price for \$XYZ\$ into a specific, pre-agreed slot in the shared buffer. 2. Reader: The Engine constantly reads the data from that same slot to trigger a trade decision.
Testing Focus	Testing must focus on synchronization and concurrency errors (e.g., deadlocks, race conditions, or one module overwriting data before the other module has finished reading it). The data structure within the shared block must be perfectly aligned between modules.

DIFFERENT TYPES OF INTERFACES AND INTERFACE ERRORS

Element	Description
Module 1 (Sender)	TicketCreationService (Responds to the user and creates a ticket record)
Module 2 (Receiver)	NotificationService (Sends emails and updates Slack channels)
Interface Channel	A Message Queue (MQ) or REST API Endpoint (e.g., RabbitMQ, SQS, or HTTPS).
Interaction	The sender sends a message/request and immediately continues its work. The receiver picks up the message when ready. 1. Sender: TicketCreationService initializes a JSON data structure ({ "ticket_id": 456, "status": "new" }). 2. Sender: It sends this JSON message to the NotificationService's designated endpoint (e.g., POST /api/notify). 3. Receiver: The NotificationService receives the message and executes its tasks (sends email, updates Slack).
Control/Data Flow	
Testing Focus	Testing must ensure the message payload (data structure) is correctly formatted and that the receiver handles the communication protocol errors (e.g., network timeout, invalid JSON structure). This also includes testing queue capacity and message loss.

DIFFERENT TYPES OF INTERFACES AND INTERFACE ERRORS

It is difficult to observe interface errors by performing system-level testing, because these errors tend to be buried in system internals?

System integration testing is performed by the **system integration group**, also known as a build engineering group. The integration test engineers need to know the details of the software modules.

This means that the team of engineers who built the modules needs to be involved in system integration.

The integration testers should be familiar with the interface mechanisms.

The **system architects** should be involved in the integration testing of complex software systems because of the fact that they have the bigger picture of the system.

Granularity of System Integration Testing

Integration testing includes both white- and black-box testing approaches.

System Integration testing is performed at different levels of granularity

Intra-system testing

- This form of testing constitutes low-level integration testing with the objective of combining the modules together to build a cohesive system

• Inter-system testing

- It is a high-level testing phase which requires interfacing independently tested systems

• Pairwise testing

- In pairwise integration, only two interconnected systems in an overall system are tested at a time
- The purpose of pairwise testing is to ensure that two systems under consideration can function together, assuming that the other systems within the overall environment behave as expected

System Integration Techniques

Common approaches to perform system integration testing

- Incremental
- Top-down
- Bottom-up
- Sandwich
- Big-bang

Pre-requisite

A module must be available to be integrated

A module is said to *available* for combining with other modules when the module's *check-in request form* is ready

Incremental

- A **software image** is a compiled software binary
- A **build is an interim** software image for internal testing within an organization
- **Constructing a build** is a process by which individual modules are integrated to form an interim software image.
- The final build is a candidate for system testing
- Constructing a software image involves the following activities
 - Gathering the latest unit tested, authorized versions of modules
 - Compiling the source code of those modules
 - Checking in the compiled code to the repository
 - **Linking the compiled modules into subassemblies**
 - **Verifying that the subassemblies are correct**
 - **Exercising version control**

Incremental

- Integration testing is conducted in an incremental manner as a series of test cycles
- In each test cycle, a few more modules are integrated with an existing and tested build to generate larger builds
- The complete system is built, cycle by cycle until the whole system is operational for system-level testing.
- The number of SIT cycles and the total integration time are determined by the following parameters:
 - Number of modules in the system
 - Relative complexity of the module
 - Relative complexity of the interfaces between the modules
 - Number of modules needed to be clustered together in each test cycle
 - Whether the modules to be integrated have been adequately tested before
 - Turnaround time for each test-debug-fix cycle

Incremental

- A release note (Status Report) containing the following information accompanies a build.
 - What has changed since the last build?
 - What outstanding defects have been fixed?
 - What are the outstanding defects in the build?
 - What new modules, or features, have been added?
 - What existing modules, or features, have been enhanced, modified, or deleted?
 - Are there any areas where unknown changes may have occurred?
- A test strategy is created for each new build and the following issues are addressed while planning a test strategy
 - What test cases need to be selected from the SIT test plan?
 - What previously failed test cases should now be re-executed in order to test the fixes in the new build?
 - How to determine the scope of a partial regression tests?
 - What are the estimated time, resource demand, and cost to test this build?

Incremental

Creating a daily build is very popular among many organization

- It facilitates to a faster delivery of the system
- It puts emphasis on small incremental testing
- It steadily increases number of test cases
- The system is tested using automated, re-usable test cases
- An effort is made to fix the defects that were found within 24 hours
- Prior version of the build are retained for references and rollback
- A typical practice is to retain the past 7-10 builds

Top-down

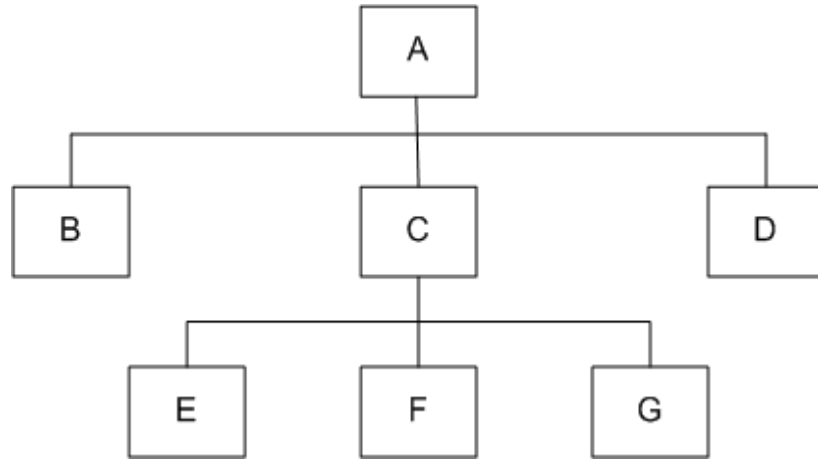


Figure 7.1: A module hierarchy with three levels and seven modules

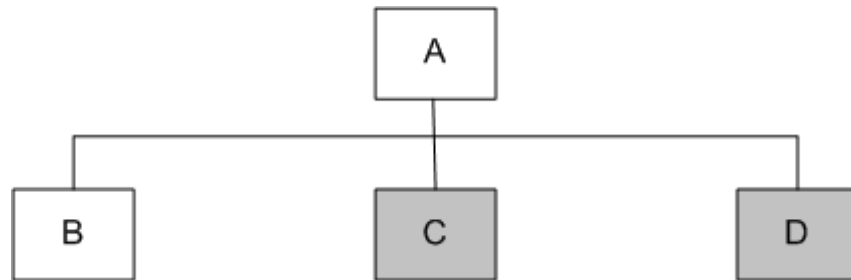


Figure 7.2: Top-down integration of modules A and B

- Module A has been decomposed into modules B, C, and D
- Modules B, D, E, F, and G are terminal modules
- First integrate modules A and B using stubs C` and D` (represented by grey boxes)
- Next stub D` has been replaced with its actual instance D
- Two kinds of tests are performed:
 - Test the interface between A and D
 - Regression tests to look for interface defects between A and B in the presence of module D

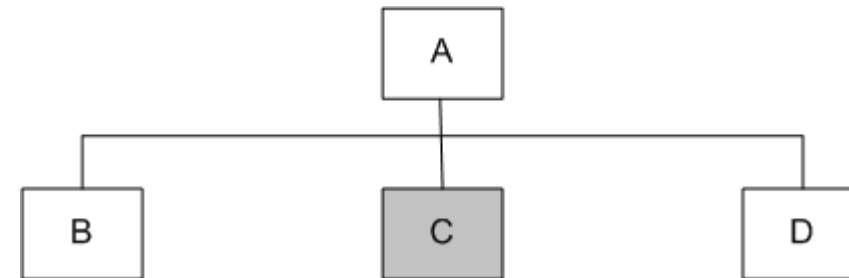


Figure 7.3: Top-down integration of modules A, B and D

Top-down

- Stub C` has been replaced with the actual module C, and new stubs E`, F`, and G`
- Perform tests as follows:
 - first, test the interface between A and C;
 - second, test the combined modules A, B, and D in the presence of C

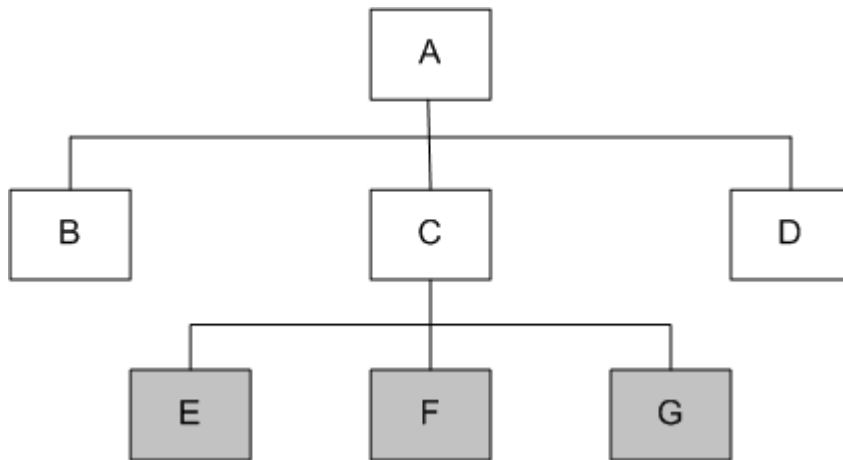


Figure 7.4: Top-down integration of modules A, B, D and C

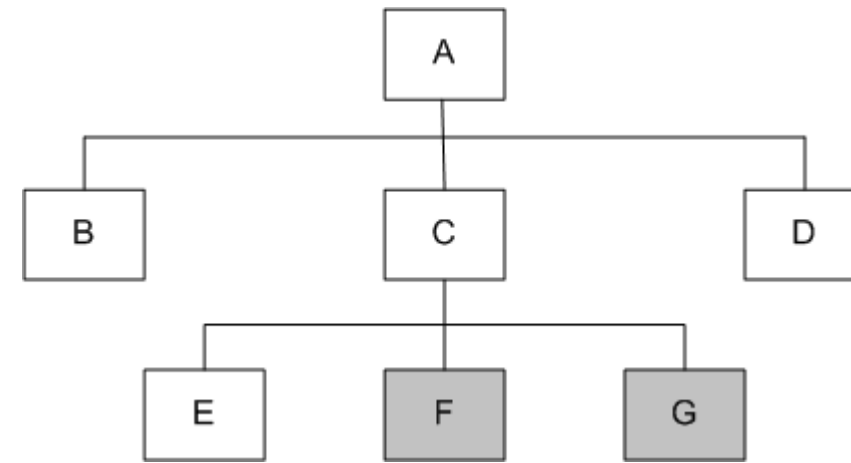


Figure 7.5: Top-down integration of modules A, B, C, D and E

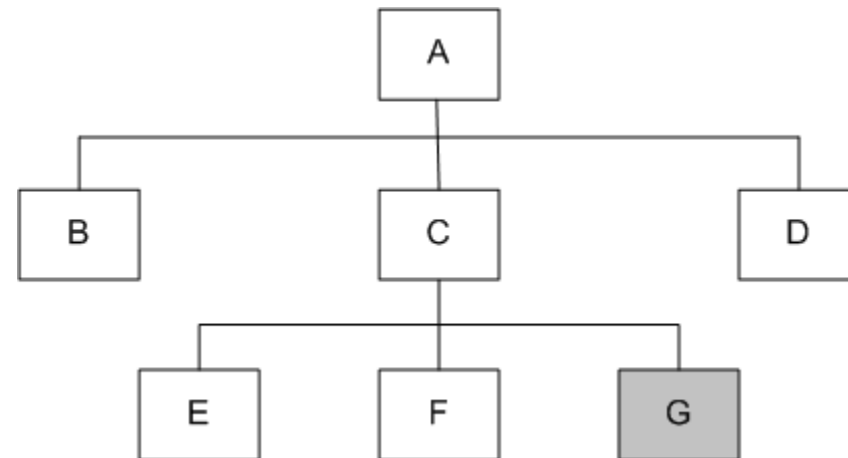


Figure 7.6: Top-down integration of modules A, B, C, D, E and F

Top-down

Advantages

- The SIT engineers continually observe system-level functions as the integration process continues
- Isolation of interface errors becomes easier because of the incremental nature of the top-down integration
- Test cases designed to test the integration of a module M are reused during the regression tests performed after integrating other modules

Disadvantages

- It may not be possible to observe meaningful system functions because of an absence of lower level modules and the presence of stubs.
- Test case selection and stub design become increasingly difficult when stubs lie far away from the top-level module.

Bottom-up

- We design a test driver to integrate lowest-level modules E, F, and G
- Return values generated by one module is likely to be used in another module
- The test driver mimics module C to integrate E, F, and G in a limited way.
- The test driver is replaced with actual module , i.e., C.
- A new test driver is used
- At this moment, more modules such as B and D are integrated
- The new test driver mimics the behavior of module A
- Finally, the test driver is replaced with module A and further test are performed

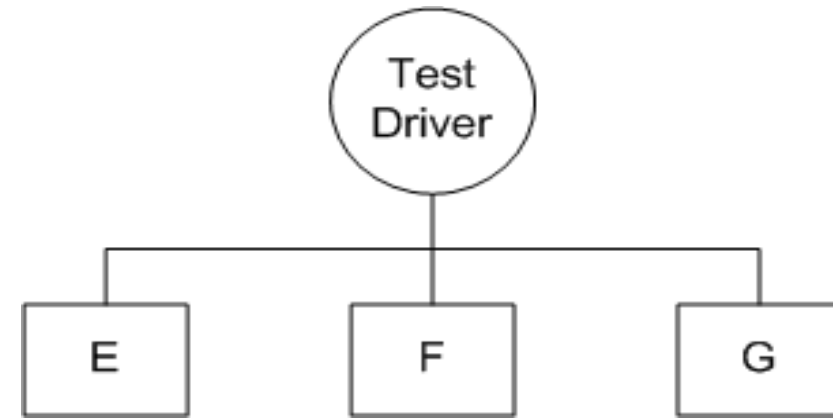


Figure 7.8: Bottom-up integration of module E, F, and G

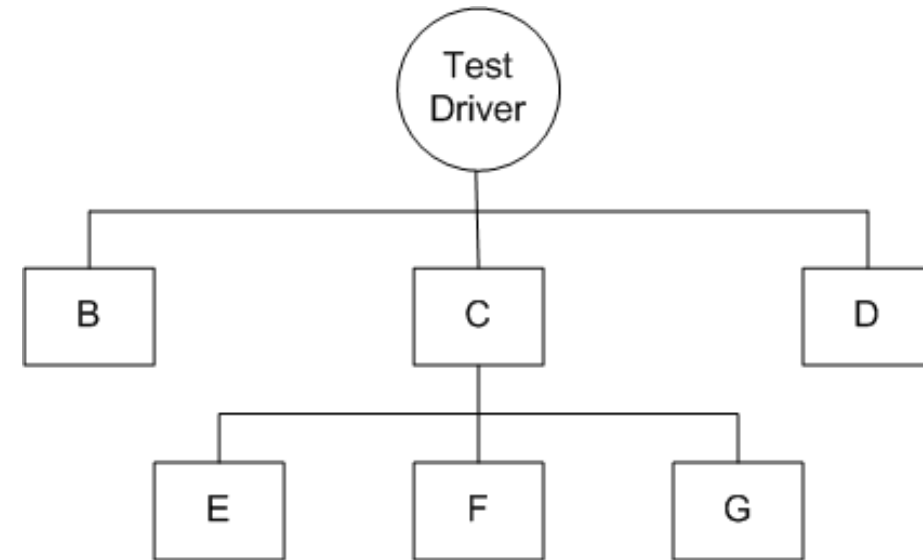


Figure 7.9: Bottom-up integration of module B, C, and D with F, F, and G

Advantages

- One designs the behavior of a test driver by simplifying the behavior of the actual module
- If the low-level modules and their combined functions are often invoked by other modules, then it is more useful to test them first so that meaningful effective integration of other modules can be done

Disadvantages

- Discovery of major faults are detected towards the end of the integration process, because major design decision are embodied in the top-level modules
- Test engineers can not observe system-level functions from a partly integrated system. In fact, they can not observe system-level functions until the top-level test driver is in place

Big-bang and Sandwich

Big-bang Approach

- First all the modules are individually tested
- Next all those modules are put together to construct the entire system which is tested as a whole

Sandwich Approach

- In this approach a system is integrated using a mix of top-down, bottom-up, and big-bang approaches
- A hierarchical system is viewed as consisting of three layers
- The bottom-up approach is applied to integrate the modules in the bottom-layer
- The top layer modules are integrated by using top-down approach
- The middle layer is integrated by using the big-bang approach after the top and the bottom layers have been integrated

TABLE 7.3 Framework for SIT Plan

1. Scope of testing
 2. Structure of integration levels
 - a. Integration test phases
 - b. Modules or subsystems to be integrated in each phase
 - c. Building process and schedule in each phase
 - d. Environment to be set up and resources required in each phase
 3. Criteria for each integration test phase n
 - a. Entry criteria
 - b. Exit criteria
 - c. Integration Techniques to be used
 - d. Test configuration set-up
 4. Test specification for each integration test phase
 - a. Test case ID number
 - b. Input data
 - c. Initial condition
 - d. Expected results
 - e. Test procedure
 - How to execute this test?
 - How to capture and interpret the results?
 5. Actual test results for each integration test phase
 6. References
 7. Appendix
-

Framework for SIT

1. **Scope of testing:** In the scope of testing section, one summarizes the system architecture.

Functional, internal, and performance characteristics

2. Structure of integration levels

1. Integration test phases

1. division of integration testing into different phases, such as functional (interface check), end-to-end, and endurance phases (reliability).

2. Modules or subsystems to be integrated in each phase

3. Building process and schedule in each phase

1. Daily, weekly, monthly build etc

4. Environment to be setup and resources required in each phase

Framework for SIT

3. Criteria for each integration test phase n

1. Entry Criteria

Start date + Table 7.4

2. Exit Criteria

End date + Table 7.4

3. Integration technique to be used

top down or bottom up

4. Test configuration setup

Framework for SIT

4. Test specification for each integration test phase

1. Test Case ID Number

2. Input Data

3. Initial Condition

4. Expected results

5. Test procedure

How to execute this test?

How to capture and interpret the results?

5. Actual test results for each integration test phase

6. References

7. Appendix

Framework for SIT

System integration testing is performed in phases of increasing complexity for better efficiency and effectiveness.

In the first phase interface integrity and functional validity within a system are tested.

In the second phase end-to-end and pairwise tests are conducted.

Finally, in the third phase, stress and endurance tests are performed.

The categories of system integration tests and the corresponding test cases discussed below are applicable for the different test phases.

The categories of system integration tests

Interface Integrity: Internal and external interfaces are tested as each module is integrated into the structure.

When two modules are integrated, the communication between them is called internal, whereas when they communicate with the outside environment, it is called external.

The number of parameters sent in a message agree with the number of parameters expected to be received. • The parameter order in the messages match the order expected. • The field sizes and the data types match. • The boundaries of each data field in a message match the expected boundaries. • When a message is generated from stored data prior to being sent, the message truly reflects the stored data. • When a received message is stored, data copying is consistent with the received message.

The categories of system integration tests

Functional Validity: Tests are designed to uncover functional errors in each module after it is integrated with the system. Errors associated with local or global data structures are uncovered with such tests. Selected unit tests that were designed for each module are reexecuted during system integration by replacing the stubs with their actual modules.

End-to-End Validity: Tests are performed to ensure that a completely integrated system works together from end to end. Interim checkpoints on an end-to-end flow provide a better understanding of internal mechanisms. This helps in locating the sources of failures.

The categories of system integration tests

Pairwise Validity: Tests are performed to ensure that any two systems work properly when connected together by a network. The difference between pairwise tests and end-to-end tests lies in the emphasis and type of test cases. For example, a toll-free call to an 800 number is an end-to-end test of a telephone system, whereas a connectivity test between a handset and local private branch exchange (PBX) is a pairwise test.

Interface Stress: Stress is applied at the module level during the integration of the modules to ensure that the interfaces can sustain the load. On the other hand, full-scale system stress testing is performed at the time of system-level testing.

The categories of system integration tests

Interface Stress: Stress is applied at the module level during the integration of the modules to ensure that the interfaces can sustain the load. On the other hand, full-scale system stress testing is performed at the time of system-level testing.

Configuration, Logging, CPU Cycles, Memory/Diskate Usage etc.

System Endurance: A completely integrated system is expected to stay up continuously for weeks without any crashes. In the case of communication protocol systems, formal rules govern that two systems communicate with each other via an interface, that is, a communication channel. The idea here is to verify that the format and the message communication across the interface of the modules works for an extended period.

Off-The-Shelf Component Integration

Instead of developing a software component from scratch, organizations occasionally purchase off-the-shelf (OTS) components from third-party vendors and integrate them with their own components

Vigder and Dean [9] have presented elements of an architecture for integration and have defined rules that facilitate integration of components.

They have identified a useful set of supporting components for integrating the actual, serving components. The supporting components are wrappers, glue, and tailoring.

Rine et al. [10] have proposed the concept of adapters to integrate components.

An adapter is associated with each component; the adapter runs an interaction protocol to manage communications among the components.

Components request services from others through their associated adapters.

An adapter is responsible for resolving any syntactic interface mismatch.

Off-The-Shelf Component Integration

A wrapper is a piece of code that one builds to isolate the underlying components from other components of the system.

Here isolate means putting restrictions around the underlying component to constrain its capabilities.

A glue component provides the functionality to combine different components.

Off-the-Shelf Component Testing

Buyer organizations perform two types of testing on an OTS component before purchasing: (i) acceptance testing of the OTS component (ii) integration of the component with other components developed in-house or purchased from a third party.

During integration testing, additional software components, such as a glue or a wrapper, can be developed to bind an OTS component with other components for proper functioning.

Off-the-Shelf Component Testing

Voas [12] proposed three types of testing techniques to determine the suitability of an OTS component:

Black-Box Component Testing: This is used to determine the quality of the component.

System-Level Fault Injection Testing: This is used to determine how well a system will tolerate a failing component. System-level fault injection does not demonstrate the reliability of the system; instead, it can predict the behavior of the system if the OTS component fails.

Operational System Testing: This kind of test is used to determine the tolerance of a software system when the OTS component is functioning correctly. Operational system testing is conducted to ensure that an OTS component is a good match for the system.

Off-the-Shelf Component Testing

Several related artefacts should be archived and/or modified for each COTS component, because a potential buyer may demand these artifacts to be of high quality

- Individual requirements, including the pointers between the software functional specification and the corresponding implementations:

The test suite, including the requirement traceability matrix:

The individual pass–fail result of each test case in the test suite:

The details of individual test cases, including input and expected

The system test report corresponding to the final release of the COTS component, which includes performance characteristics, scalability limitation, stability observations, and interoperability of the COTS component: